

Tema 1: Metaheurísticas para Optimización

José Luis Ruiz Reina
Miguel A. Gutiérrez Naranjo
Agustín Riscos Núñez

Departamento de Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

Inteligencia Artificial

Índice

- Problemas de optimización y metaheurísticas
- Escalada y enfriamiento simulado
- Algoritmos genéticos
- PSO

Optimización

- En un conjunto $\mathcal{O}$, hay una función $F : \mathcal{O} \rightarrow \mathbb{R}$
 - F cuantifica cómo de bueno es cada elemento de $\mathcal{O}$
 - Terminología: valoración, función objetivo, *fitness*...
- Optimización: encontrar un elemento $x \in \mathcal{O}$ con la mejor valoración $F(x)$
 - Si se busca el de menor valoración: minimización
 - Si se busca el de mayor valoración: maximización
- Muchos de los problemas que se plantean en Inteligencia Artificial son en realidad problemas de optimización.
- Problema computacional: el conjunto $\mathcal{O}$ suele ser **enormemente grande**
 - Por tanto, no nos vale un simple algoritmo de *fuerza bruta*, que recorra *todo el conjunto* $\mathcal{O}$
 - Tenemos que recurrir a *técnicas heurísticas*

Ejemplo clásico: problema del viajante

- Con 8 ciudades, existen 2520 posibles rutas (es abordable).
- Con 100 ciudades, hay $\frac{99!}{2}$ posibles rutas
 - Más de 10^{155} posibles rutas candidatas.
 - Es **completamente inabordable** encontrar la ruta óptima usando métodos exhaustivos y completos, en un tiempo razonable¹

¹Un súper-ordenador que evalúe 10^{18} rutas por segundo (lo normal serían unas 10^7) tardaría unos 10^{130} años.

Heurística

- **Heurística:** en Computación, técnica para resolver problemas de manera eficiente, a partir de conocimiento específico sobre el problema
 - Usualmente, soluciones aproximadas, métodos no completos
 - Pero razonablemente buenos en la mayoría de los casos
- Ejemplos:
 - Reglas para clasificar SPAM
 - Reglas para jugar a juegos de estrategia
 - Técnicas para problemas de optimización: viajante, mochila, . . .

Metaheurística

- Cuando la técnica es heurística, pero lo suficientemente genérica como para poder aplicarla a problemas de distinto tipo, la denominamos **metaheurística**
- En este tema, veremos las siguientes técnicas **metaheurísticas para optimización**:
 - Escalada y enfriamiento simulado
 - Algoritmos genéticos
 - PSO: *Particle Swarm Optimization*

Mejoras iterativas

- La mayoría de las técnicas que veremos en este tema se basan en la idea de *mejora iterativa*:
 - Empezar con un(os) candidato(s) inicial(es) cual(es)quiera
 - Mejorar su calidad paso a paso
- Búsqueda local
- “Mejorar”:
 - Usualmente con una componente aleatoria, probabilística
 - Heurística basadas en conocimiento previo sobre el problema
- Problema de los *óptimos locales*

Búsqueda en espacio de estados vs búsqueda local

- Búsqueda en espacio de estados:
 - Objetivo: Encontrar una sucesión ordenada de acciones o decisiones que vaya transformando una situación de partida (estado inicial) en otra que verifique una serie de condiciones pre-establecidas (estado final, no tiene por qué ser único).
 - Método: Explorar el espacio de estados construyendo un árbol (ej. BFS, DFS, A*, ...) que debe mantener en memoria.
- Búsqueda local:
 - Objetivo: Encontrar la mejor solución dentro del conjunto completo de soluciones O.
 - Método: Explorar el conjunto de soluciones pasando de unas a otras realizando pequeños cambios. Sólo mantiene en memoria las soluciones que esté analizando en cada momento.

Características de algunos problemas de optimización

- Estado $\equiv$ solución al problema
 - Todos los estados son soluciones candidatas
 - Se busca la solución con mejor **valoración** (máximo o mínimo): óptimo
- Estado inicial: no está claramente definido
- Operadores:
 - Se puede definir cierta noción de estado “sucesor” o “vecino”
 - En algunos casos, gran cantidad de vecinos
 - Introducimos cierta componente heurística y aleatoria, generando cada vez un único “estado vecino”

Representación de un problema de optimización (para aplicar búsqueda local)

Elegir una representación para los estados (estructura de datos)

genera_estado_inicial()

- Debe generarse con alguna componente aleatoria (puede usarse alguna técnica heurística)

genera_sucesor(estado)

- Genera un estado sucesor parecido a uno dado
- Define la noción de “vecindad” para el problema concreto
- Existe cierta componente aleatoria (puede usarse alguna técnica heurística)

valoracion(estado)

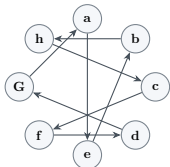
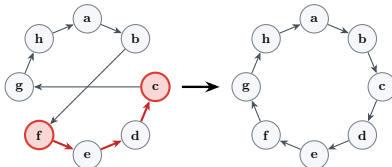
- Calidad del estado, métrica que se trata de optimizar (minimizar o maximizar)

Especificación del problema del viajante

Estado: un recorrido con *todas* las ciudades (permutación). Tras visitar la última ciudad se vuelve a la primera.

- **genera_estado_inicial()**: obtiene un recorrido aleatorio
- **genera_sucesor(estado)**: elige dos ciudades distintas e invierte el recorrido entre ellas
 - Heurística: trata de reducir los “cruces”
 - Aleatoriedad: al elegir el par de ciudades
- **valoracion(estado)** devuelve la distancia total del recorrido

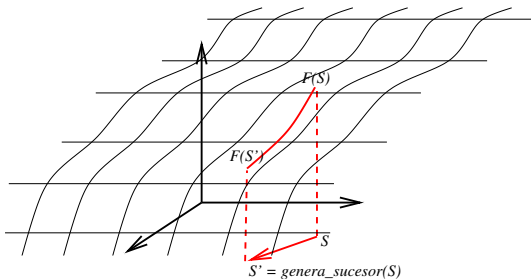
```
genera_estado_inicial()
```


$$S_0 = [a, e, b, h, c, f, d, g]$$
$$F(S_0) = 425 \text{ km}$$
$$\text{genera_sucesor}(S)$$

$$S = [a, b, \mathbf{f}, \mathbf{e}, \mathbf{d}, \mathbf{c}, g, h]$$
$$F(S) = 310 \text{ km}$$
$$S' = [a, b, c, d, e, f, g, h]$$
$$F(S') = 215 \text{ km}$$

Escalada

Idea de escalada:

- Aplicar una simple *mejora iterativa*
- Guiado por la heurística y aleatoriedad de la generación de sucesores
- No se recupera tras una elección errónea
- Puede verse como una escalada (ascenso o descenso)



Pseudocódigo: algoritmo de escalada

escalada()

1. `actual` $\leftarrow$ `genera_estado_inicial()`
 `valor_actual` $\leftarrow$ `valoracion(actual)`
 `candidata` $\leftarrow$ `genera_sucesor(actual)`
 `valor_candidata` $\leftarrow$ `valoracion(candidata)`
2. Mientras `valor_candidata` sea mejor que `valor_actual`:
 - 2.1 `actual` $\leftarrow$ `candidata`
 `valor_actual` $\leftarrow$ `valor_candidata`
 `candidata` $\leftarrow$ `genera_sucesor(actual)`
 `valor_candidata` $\leftarrow$ `valoracion(candidata)`
3. Devolver `actual` y `valor_actual`

Escalada: ejemplo

Viajante en Andalucía

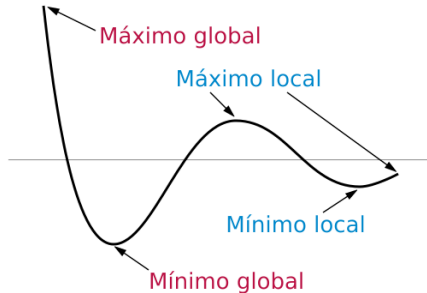
Ejemplos de resultados obtenidos experimentalmente:

Recorrido obtenido	Distancia
[CA, HU, CO, JA, AL, SE, GR, MA]	1366.7968 Km
[SE, JA, HU, CO, MA, AL, GR, CA]	1488.9255 Km
[SE, GR, AL, CO, JA, CA, MA, HU]	1431.388 Km



Escalada: conclusiones

- Fortalezas:
 - Muy eficiente en tiempo y en espacio
- Debilidades:
 - Puede no encontrar solución óptima si encuentra óptimos locales o mesetas



Escalada: conclusiones

- Su eficacia depende en gran medida de `genera_sucesor`
- Una idea simple para intentar escapar de los óptimos locales:
 - Buscar aleatoriamente el inicio de una pendiente
 - Hacer escalada (ascenso o descenso) a partir de ahí
 - Iterar el proceso
 - Devolver el mejor estado conseguido en alguna de las iteraciones
- A mayor número de iteraciones, es esperable mejor resultado final (aunque no está garantizado)

Pseudocódigo: algoritmo de escalada con reinicio aleatorio

`escalada_reinicio_aleatorio(n_iter)`

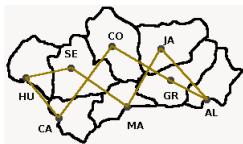
1. `mejor_estado` $\leftarrow$ `genera_estado_inicial()`
 `mejor_valor` $\leftarrow$ `valoracion`(`mejor_estado`)
2. Repetir `n_iter` veces:
 - 2.1 `actual` y `valor_actual` $\leftarrow$ `escalada()`
 - 2.2 Si `valor_actual` es mejor que `mejor_valor`,
 - 2.2.1 `mejor_estado` $\leftarrow$ `actual`
 - `mejor_valor` $\leftarrow$ `valor_actual`
3. Devolver `mejor_estado` y `mejor_valor`

Escalada con reinicio aleatorio: ejemplo

Viajante en Andalucía

Ejemplos de resultados obtenidos experimentalmente:

n_iter	Recorrido obtenido	Distancia
50	[JA, CO, SE, CA, HU, MA, AL, GR]	1009.58923 Km
300	[CO, GR, AL, JA, MA, SE, HU, CA]	1080.9673 Km
1000	[MA, CA, HU, SE, CO, JA, AL, GR]	929.9256 Km



- La última es **óptima**

Búsqueda en escalada con reinicio aleatorio: ejemplos

Problema del cuadrado de puntos ($n = 3$)

Ejemplos de resultados obtenidos experimentalmente:

<code>n_iter</code>	Recorrido obtenido	Distancia
10000	[(3, 3), (3, 2), (2, 3), (3, 1), (2, 0), (3, 0), (1, 0), (0, 0), (0, 1), (0, 3), (0, 2), (1, 3)]	17.478706
100000	[(1, 0), (0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (3, 2), (3, 3), (2, 3), (3, 0), (3, 1), (2, 0)]	15.812559

- Aún es necesario mejorar la técnica para evitar óptimos locales

Enfriamiento/templado simulado

Idea principal:

- Aceptar probabilísticamente estados “peores”: **minimización**
- La probabilidad de que un estado peor sea aceptado varía en función del incremento producido en la valoración
- Permitimos así salir de óptimos locales, sin salir del óptimo global

Algoritmo inspirado en el proceso físico-químico de *enfriamiento de metales*

Enfriamiento simulado

(inspiración físico-química)

En metalurgia, el proceso de *enfriamiento de metales* se utiliza para templar o endurecer metales y vidrio, calentándolos a altas temperaturas para después irlos enfriando gradualmente, permitiendo que el material alcance un estado cristalino de muy baja energía.

- Sistema estable: **mínimo** de energía
- Al bajar la temperatura ambiente, el sistema necesita tiempo para estabilizarse (perder energía)
- Es posible pasar momentáneamente por estados de mayor energía, con probabilidad dada por

$$p(\Delta F, T) = e^{-\frac{\Delta F}{T}}$$

- Después de estabilizarse, se vuelve a bajar la temperatura, y el sistema se estabiliza nuevamente en un estado de menor energía

Enfriamiento simulado

Programa de enfriamiento

- Decidimos cuántos descensos de temperatura vamos a realizar
- Al principio (T grande) mayor probabilidad de aceptación de soluciones candidatas (diversificación)
- Al final (T pequeña), se aceptan pocas soluciones candidatas (intensificación)

Enfriamiento simulado

- Generación del estado inicial y estados vecinos
 - Heurística + Aleatoriedad
- Aceptación probabilística
 - Probabilidad de aceptación de un estado según el incremento en el valor de la valoración, ΔF : $e^{-\frac{\Delta F}{T}}$
- Programa de enfriamiento, en nuestro caso:
 - Temperatura inicial
 - Variación de la temperatura: $T \leftarrow \alpha \cdot T$
 - Número fijo de iteraciones para cada T
- Criterio de parada
 - Valor suficientemente bueno de la valoración (umbral)
 - Número máximo de iteraciones sin mejora
 - En nuestro caso, número fijo de iteraciones totales

Pseudocódigo: algoritmo de enfriamiento simulado

minimización

enfriamiento_simulado($t_{inicial}$, α , n_{enfr} , n_{iter})

1. $actual \leftarrow genera_estado_inicial()$
 $valor_actual \leftarrow valoracion(actual)$
 $mejor \leftarrow actual$
 $valor_mejor \leftarrow valor_actual$
 $T \leftarrow t_{inicial}$
2. Repetir n_{enfr} veces:
 - 2.1 Repetir n_{iter} veces:
 - 2.1.1 $candidata \leftarrow genera_sucesor(actual)$
 $valor_candidata \leftarrow valoracion(candidata)$
 $\Delta F \leftarrow valor_candidata - valor_actual$
 - 2.1.2 Si $\Delta F < 0$ ó
 se acepta (con probabilidad de aceptación $e^{-\frac{\Delta F}{T}}$)
 - 2.1.2.1 $actual \leftarrow candidata$
 $valor_actual \leftarrow valor_candidata$
 - 2.1.3 Si $valor_actual < mejor_valor$
 - 2.1.3.1 $mejor \leftarrow actual$
 $valor_mejor \leftarrow valor_actual$
 - 2.2 $T \leftarrow \alpha \cdot T$
3. Devolver $mejor$ y $mejor_valor$

Enfriamiento simulado: ejemplos

- parámetros: `t_inicial`, α , `n_enfr`, `n_iter`

Viajante (Andalucía)

- parámetros: 100, 0.95, 100, 100 $\Rightarrow$ 929.9256 Km (*óptimo*)
[MA, GR, AL, JA, CO, SE, HU, CA,]

Viajante (Cuadrado de puntos)

- 12 puntos
 - parámetros: **5**, 0.95, 100, 100 $\Rightarrow$ 12 (*óptimo*)
- 28 puntos
 - parámetros: **15**, 0.95, 100, 100 $\Rightarrow$ 28 (*óptimo*)
- 60 puntos
 - parámetros: **35**, 0.95, 100, 100 $\Rightarrow$ 99.498474
 - parámetros: 35, 0.95 **500**, **500** $\Rightarrow$ 60 (*óptimo*)

Algoritmos genéticos: evolución natural

- Optimización inspirada en los procesos evolutivos de la naturaleza:
 - La evolución ocurre en los cromosomas de los individuos
 - Las “buenas estructuras” sobreviven con más probabilidad que las demás
 - El nuevo material genético se obtiene mediante cruces y mutaciones
- Algoritmos genéticos:
 - Aplicación de estas ideas en la búsqueda de soluciones óptimas
 - No existe un único algoritmo genético
 - Es una denominación para este tipo de algoritmos evolutivos

Algoritmos genéticos: codificación del problema

- Un primer paso es representar los estados/soluciones del problema como cromosomas/individuos de una población
 - Genes: material genético básico
 - Cromosomas: sucesión ordenada de genes que codifica a un estado/solución del problema
 - Población: conjunto de cromosomas (sólo un subconjunto de todos los posibles de tamaño “manejable”)
 - Se trata de evitar los óptimos locales manejando una población de candidatos, en lugar de un único candidato
 - La población evoluciona a lo largo de distintas *generaciones*
 - Genotipo (el cromosoma) vs fenotipo (solución que representa el cromosoma)
- Bondad de los cromosomas
 - Según el valor de la función objetivo (usualmente llamada *fitness*)

Representación de un problema de optimización (para aplicar un algoritmo genético)

- Constantes: **genes** y **longitud_cromosoma**
 - Los “estados” o “soluciones” del problema se representan por “cromosomas”: una sucesión ordenada (con o sin repeticiones) de “genes” de “tamaño” fijo
- **decodifica(cromosoma)**
 - obtiene el *fenotipo* (una “traducción” de la solución que representa el cromosoma)
- **fitness(cromosoma)**
 - métrica que se trata de optimizar (usualmente calcula primero el fenotipo)

Especificación del problema mínimo de la función cuadrado

Hallar el mínimo de la función $g(x) = x^2$ con $x \in [0, 2^{10}) \cap \mathbb{N}$

- **genes**: $\{0, 1\}$
- **longitud_cromosoma**: 10
- **decodifica**(cromosoma): el cromosoma puede verse como un número binario de 10 dígitos (en orden inverso). El fenotipo de un cromosoma es dicho número en notación decimal.
Ejemplo: el cromosoma $[0, 1, 1, 0, 0, 1, 0, 0, 0, 0]$ representa al número $\sum_{g_i=1} 2^i = 2^1 + 2^2 + 2^5 = 38$
- **fitness**(cromosoma): la función g sobre el fenotipo
Ejemplo: $[0, 1, 1, 0, 0, 1, 0, 0, 0, 0]$ es un cromosoma cuya *fitness* es $38^2 = 1444$

Especificación del problema del viajante en Andalucía

Encontrar el recorrido mínimo (en cuanto a distancia) por todas las ciudades de Andalucía

- **genes**: {*AL*, *CA*, *CO*, *GR*, *HU*, *JA*, *MA*, *SE*}
- **longitud_cromosoma**: 8
- **decodifica**(**cromosoma**) El fenotipo de un cromosoma es recorrer las ciudades en el orden en el que aparecen.
Considerar, únicamente, cromosomas en los que los genes no se repitan
- **fitness**(**cromosoma**) Distancia del viaje al recorrer las ciudades en el orden en el que aparecen

Algoritmos genéticos: mecanismo evolutivo

- Evolución de las sucesivas generaciones
 - Mecanismos de selección
 - Sirven tanto para decidir qué cromosomas se van a usar para combinación, como para decidir cuáles pasan a la generación siguiente
 - En general, debe ser un método de selección que favorezca a los cromosomas con mejor *fitness*, pero que no impida la *diversidad*
 - Operadores de variación:
 - Cruces: combinación de cromosomas (*padres* que dan lugar a *hijos*)
 - Mutaciones: cambio de algunos genes
- En general, es bastante común que se maneje aleatoriedad

Esquema general de un algoritmo genético

Iniciar generación

Repetir hasta **condición de terminación**

Seleccionar padres de la generación

Cruzar pares de padres ("nuevos" cromosomas: hijos)

Mutar hijos

Seleccionar cromosomas para la siguiente generación

Devolver el fenotipo del mejor cromosoma de generación

Condiciones de terminación

- Alcanzar un número de generaciones (repeticiones) máximo pre-establecido.
- No se produce una mejora significativa entre una generación y la(s) anterior(es).
- Se alcanza un valor umbral de *fitness* pre-establecido.

Selección, reproducción y mutación

- Selección de padres:
 - Un número fijo de padres se seleccionan de cada generación con preferencia por los mejores
 - El resto se escoge aleatoriamente (para mantener así la *diversidad*)
- Cruces:
 - Los hijos se obtienen cruzando los padres de dos en dos
- Mutación:
 - Cada gen de cada cromosoma se mutará según una probabilidad dada

Atención: existen muchas variantes de algoritmos genéticos

- Todas se basan en lo mismo: selección de los mejores, mantenimiento de la diversidad, reproducción y mutación

Mecanismos de selección

- Un algoritmo genético debe tener un método para seleccionar cromosomas de una generación:
 - Para obtener aquellos cromosomas que van a ser usados como padres
 - A veces, también para decidir qué cromosomas pasan a la siguiente generación
- La selección debe:
 - Favorecer a los mejores (según su *fitness*)
 - Permitir la diversidad
 - Usualmente tiene una componente aleatoria
- Ejemplos de métodos de selección:
 - Torneo
 - Elitista
 - Proporcional a su *fitness*
 - Proporcional a su *ranking*

Selección por torneo y elitista

- Selección por torneo:
 - Para cada selección, extraer aleatoriamente k cromosomas y seleccionar el mejor
 - Ventajas: no depende de la magnitud de la función del *fitness* y se puede aplicar tanto a minimización como a maximización
 - Cuanto mayor el k usado, mayor es la *presión evolutiva* (mayor probabilidad de que los mejores de cada generación sean seleccionados, y menor de que lo sean los peores)
- Selección elitista:
 - Escoger directamente los mejores

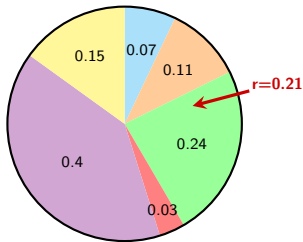
Selección por ruleta

- La probabilidad de ser seleccionado es directamente proporcional al *fitness* del cromosoma

$$p_i = \frac{\text{fitness}(c_i)}{\sum_{c \in \text{generación}} \text{fitness}(c)}$$

c_i	$F(c_i)$	p_i	P_i
A	8	0.07	0.07
B	12	0.11	0.18
C	27	0.24	0.42
D	4	0.03	0.45
E	45	0.40	0.85
F	17	0.15	1.00

113



A B C D E F

Importante: con este método de selección sólo podemos resolver problemas de maximización

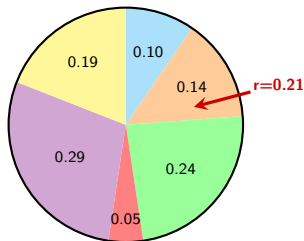
Selección por rango

(variante de la anterior)

- la probabilidad de ser seleccionado es directamente proporcional a su *ranking* (posición en la generación ordenada según *fitness*)

c_i	$F(c_i)$	Rank.	p_i	P_i
A	8	2	0.09	0.09
B	12	3	0.14	0.23
C	27	5	0.24	0.47
D	4	1	0.05	0.52
E	45	6	0.29	0.81
F	17	4	0.19	1.00

21



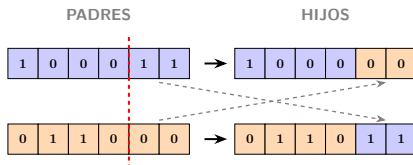
■ A ■ B ■ C ■ D ■ E ■ F

- Si varios copan los mejores valores, los acerca al resto
- Si la mayoría tiene valores similares, los separa entre sí

Cruce de cromosomas

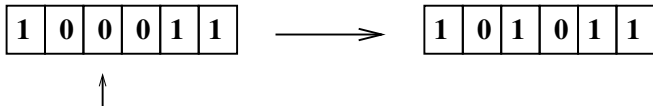
Operadores que combinan la información de los *padres* para obtener nuevos *hijos*

- Cruce en un punto:



- Aleatoriedad: al elegir el punto de cruce
- Otras posibilidades:
 - Cruce multipunto (varios segmentos de intercambio)
 - Cruce uniforme (para cada posición del hijo, se *sortea* de quién hereda)
 - Cruces específicos de la representación (p.ej. permutaciones)

Mutación de individuos



- Aleatoriedad:
 - Con una determinada probabilidad (usualmente baja) cambiar algunos genes
 - Si se cambia, elegir aleatoriamente a qué gen se cambia
- Variantes:
 - Específicas de la representación (p.ej. permutaciones)

Permutaciones

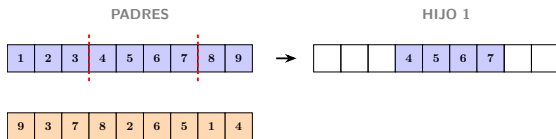
Si cada cromosoma es una permutación del conjunto de genes, es necesario tener operadores específicos de cruce y mutación.

Cruce basado en orden

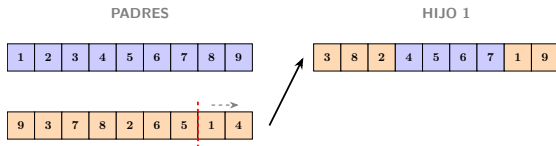
- Paso 1: Elige dos puntos de corte aleatoriamente del primer padre y copia el segmento entre ellos en el primer hijo.
- Paso 2: A partir del segundo punto de corte en el segundo padre, copia en el primer hijo los genes no usados en el mismo orden en que aparecen en el segundo padre, volviendo al principio del cromosoma si fuera necesario.
- Crea el segundo hijo de manera análoga, intercambiando el rol de los padres.

Permutaciones: Cruce basado en orden

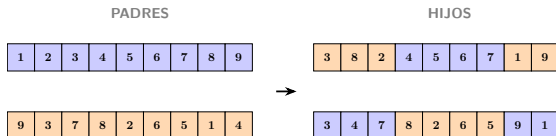
Paso 1:



Paso 2:



Resultado:



Permutaciones: Cruce basado en ciclos (I)

Dividimos los padres en ciclos y construimos los hijos alternando los ciclos de cada padre.

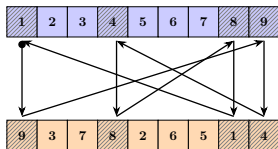
Para construir un ciclo:

1. Tomamos la primera posición *nueva* en el padre P_1 .
2. Buscamos el gen en la misma posición de P_2 .
3. Vamos a la posición con el mismo gen en P_1 .
4. Añadimos este gen al ciclo.
5. Repetimos los pasos del 2 al 4 hasta que lleguemos al primer gen de P_1 .

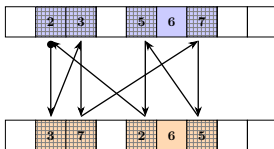
Permutaciones: Cruce basado en ciclos (II)

División de los padres en ciclos:

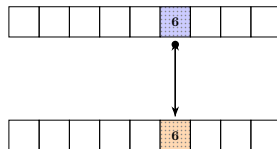
Ciclo 1



Ciclo 2



Ciclo 3

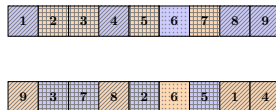


Construcción de hijos:

Padres



Hijos



Permutaciones: Mutaciones

- Mutación por intercambio:



- Mutación por inserción:



- Mutación por mezcla:



Otras componentes de un algoritmo genético

- Población inicial
 - Usualmente los cromosomas que la componen se generan de manera completamente aleatoria
- Diversos parámetros:
 - Tamaño de la población: no varía de una generación a otra
 - Proporción de la población que se selecciona como padres
 - Probabilidad de mutación

Pseudocódigo: algoritmo genético

generación $\leftarrow$ población.inicial(N)

Repetir $nGen$ veces:

 padres $\leftarrow$ Seleccionar $N \cdot p$ cromosomas de generación

 hijos $\leftarrow$ Cruzar padres

 directos $\leftarrow$ Seleccionar $N \cdot (1-p)$ cromosomas de generación

 generación $\leftarrow$ Mutar hijos y directos

Devolver el fenotipo del mejor cromosoma de poblacion

- Parámetros del algoritmo: tamaño de las poblaciones N , proporción de padres p , probabilidad de mutación, número de generaciones $nGen$
- Selección por torneo de tamaño k
- Cruce en un punto

Algoritmo genético: ejemplo (mínimo de la función cuadrado)

```
>>> algoritmo_genetico(cuad_gen, 10, 0.7, 0.05, 20, 4, min)
Generacion: 1. Media: 182192.4. Fitness mejor:42849
Generacion: 2. Media: 80760.9. Fitness mejor:30276
Generacion: 3. Media: 34026.2. Fitness mejor:5041
Generacion: 4. Media: 24990.0. Fitness mejor:4900
Generacion: 5. Media: 7550.6. Fitness mejor:4900
Generacion: 6. Media: 8344.5. Fitness mejor:4900
Generacion: 7. Media: 4872.4. Fitness mejor:4624
Generacion: 8. Media: 15375.2. Fitness mejor:4624
Generacion: 9. Media: 30381.6. Fitness mejor:16
Generacion: 10. Media: 2320.0. Fitness mejor:16
Generacion: 11. Media: 16.0. Fitness mejor:16
Generacion: 12. Media: 184.4. Fitness mejor:16
Generacion: 13. Media: 18.9. Fitness mejor:16
Generacion: 14. Media: 55.3. Fitness mejor:16
Generacion: 15. Media: 1756.8. Fitness mejor:16
Generacion: 16. Media: 26640.0. Fitness mejor:16
Generacion: 17. Media: 182.4. Fitness mejor:16
Generacion: 18. Media: 16.4. Fitness mejor:0
Generacion: 19. Media: 59608.0. Fitness mejor:0
Generacion: 20. Media: 974.4. Fitness mejor:0
```

Algoritmo genético: ejemplo (problema de la mochila)

Dados n objetos de pesos $\{p_i\}$ y valores $\{v_i\}$ $i = 1, \dots, n$, seleccionar cuáles se meten en una mochila que soporta un peso P máximo, de manera que la suma de los valores de los objetos elegidos sea la mayor posible.

- **genes:** $\{0, 1\}$
- **longitud_cromosoma:** $= n$

Algoritmo genético: ejemplo (problema de la mochila)

- **decodifica**(cromosoma)
 - Cada gen, **1** ó **0**, representa, respectivamente, si el objeto se introduce o no en la mochila
 - Recorridos de izquierda a derecha, a partir del primer objeto que se vaya a introducir con el que se supere el peso máximo permitido se considera que están todos fuera de la mochila, *independientemente del gen que aparezca en las restantes posiciones*
 - Todos los cromosomas representan soluciones, mejores o peores, del problema
- **fitness**(cromosoma)
 - Suma de valores de los objetos, que según la solución dada por la función **decodifica** anterior, están dentro de la mochila

Algoritmos Genéticos

Aplicaciones

Journal List > Croat Med J > v.60(2); 2019 Apr > PMC6509630

<i>Free full text at www.cmj.hr</i>				 CROATIAN MEDICAL JOURNAL
Info for Authors	Submit	Subscribe	About	

[Croat Med J](#). 2019 Apr; 60(2): 177–180.

doi: [10.3325/cmj.2019.60.177](https://doi.org/10.3325/cmj.2019.60.177)

PMCID: PMC6509630

PMID: [31044592](https://pubmed.ncbi.nlm.nih.gov/31044592/)

Genetics without genes: application of genetic algorithms in medicine

[Branimir K. Hackenberger](#)

Algoritmos Genéticos

Aplicaciones

npj | Computational Materials

Article | [Open Access](#) | Published: 10 April 2019

Genetic algorithms for computational materials discovery accelerated by machine learning

Paul C. Jennings, Steen Lysgaard, Jens Strabo Hummelshøj, Tejs Vegge  & Thomas Bligaard

Algoritmos genéticos: conclusiones

- Algoritmos genéticos como proceso de búsqueda local
 - Mejora iterativa
 - La selección, los cruces, las mutaciones tratan de evitar el problema de los óptimos locales
- Existen muchas variantes de algoritmos genéticos
 - Todas se basan en lo mismo: selección de los mejores, mantenimiento de la diversidad, reproducción y mutación
- Fácil de aplicar a muchos tipos de problemas:
 - optimización, aprendizaje automático, planificación,...
- Resultados aceptables en muchos problemas
 - No son mejores que algoritmos específicos para cada problema

Optimización de enjambre de partículas

Particle Swarm Optimization (PSO)

- Inspirado en el vuelo de una bandada de pájaros.
- Técnica relacionada con el vuelo de murciélagos en *Batman Returns*² o la estampida de bisontes en *El Rey León*.
- Se enmarca en el modelo conocido como *Inteligencia de Enjambre (Swarm Intelligence)*, que estudia el comportamiento colectivo de sistemas descentralizados auto-organizativos.

²I.V. Kerlow. The Art of 3D: Computer Animation and Effects. Wiley John & Son, 2009. p23.

Optimización de enjambre de partículas

Un sistema PSO consta de n partículas que ocupan una posición en $\mathbb{R}^m$ y que llevan asociado un vector de velocidad, junto con una función *fitness* que asocia un valor a cada posición. El objetivo es encontrar la posición donde la función *fitness* toma el valor mínimo (o máximo).

- La posición y velocidad inicial de las partículas se establece aleatoriamente.
- Todas las partículas actualizan su posición en paralelo.
- Todas las partículas se rigen por las mismas reglas.
- Para cada partícula, su posición en el instante $t + 1$, depende de su posición en el instante t , más la velocidad de la partícula en el instante $t + 1$

$$\vec{x}_i^{t+1} = \vec{x}_i^t + \vec{v}_i^{t+1}$$

Optimización de enjambre de partículas

$$\vec{x}_i^{t+1} = \vec{x}_i^t + \vec{v}_i^{t+1}$$

Por tanto, para conocer la nueva posición $\vec{x}_i^{t+1}$ de la partícula i , tenemos que determinar $\vec{v}_i^{t+1}$. Esa velocidad depende de tres factores:

- La velocidad $\vec{v}_i^t$ que lleva la partícula en el paso t (inercia).
- La mejor posición $\vec{m}_i^t$ que haya obtenido la partícula i en los pasos anteriores (influencia *individual* en el movimiento).
- La mejor posición que haya obtenido *cualquier* partícula en los pasos anteriores (influencia *social* en el movimiento).

Intuición: Una posición es mejor que otra si hay más alimento o el animal está más protegido. Lo medimos con una función *fitness*.

Optimización de enjambre de partículas

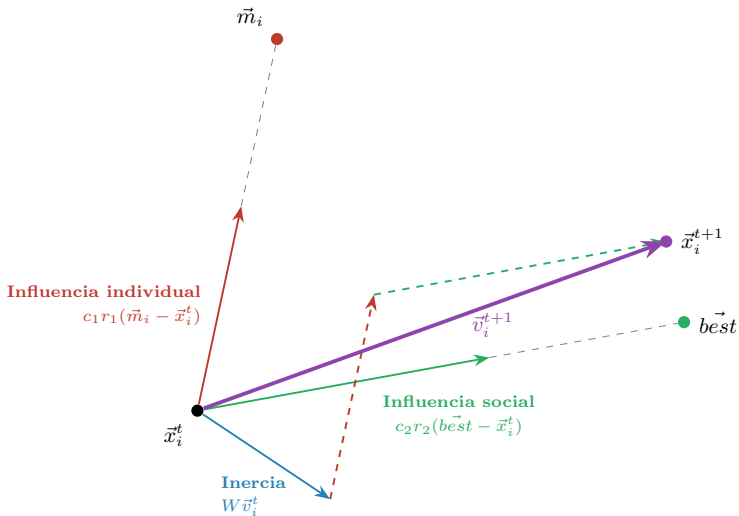
Para el cálculo de $\vec{v}_i^{t+1}$ necesitamos:

- Tres parámetros:
 - $W \in \mathbb{R}$ que pondera la inercia.
 - c_1 que pondera la influencia individual
 - c_2 que pondera la influencia social
- Dos valores aleatorios $r_1, r_2 \in [0, 1]$

$$\vec{v}_i^{t+1} = W \cdot \vec{v}_i^t + c_1 \cdot r_1 \cdot (\vec{m}_i^t - \vec{x}_i^t) + c_2 \cdot r_2 \cdot (\vec{best}^t - \vec{x}_i^t)$$

- $\vec{m}_i^t$ es la mejor posición de la partícula i desde el inicio
- $\vec{best}^t$ es la mejor posición de cualquier partícula desde el inicio

Optimización de enjambre de partículas



Pseudocódigo: algoritmo PSO

PSO(n , W , c_1 , c_2)

1. Inicializar n partículas $\vec{x}_i$ y velocidades $\vec{v}_i$
2. Para cada partícula:
 - 2.1 $\vec{m}_i \leftarrow \vec{x}_i$
3. $\vec{best} \leftarrow \text{mejor } \vec{m}_i$
4. Repetir hasta condición de terminación
 - 4.1 Para cada partícula:
 - 4.1.1 Generar r_1, r_2 (aleatorios entre 0 y 1)
 - 4.1.2 $\vec{v}_i \leftarrow W \vec{v}_i + c_1 r_1 (\vec{m}_i - \vec{x}_i) + c_2 r_2 (\vec{best} - \vec{x}_i)$
 $\vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i$
 - 4.2 Para cada partícula:
 - 4.2.1 Si $\text{fitness}(\vec{x}_i)$ es mejor que $\text{fitness}(\vec{m}_i)$
 - 4.2.1.1 $\vec{m}_i \leftarrow \vec{x}_i$
 - 4.3 $\vec{best} \leftarrow \text{mejor } \vec{m}_i$
5. Devolver $\vec{best}$

Optimización de enjambre de partículas

Ejemplo

- Optimización en $\mathbb{R}^2$
- Función fitness $f(x, y) = (x - 0,9)^2 + (y - 0,9)^2$, con mínimo en $(0,9, 0,9)$.
- $W = 0,5$, $c_1 = 0,5$, $c_2 = 0,5$
- 50 partículas. Las posiciones iniciales toman valores en $[0, 1]^2$ y las velocidades iniciales en $[-1, 1]^2$.
- Evolución de la mejor partícula:

Paso 0 – Pos: (0.7568, 0.9482) – F: 0.0228

Paso 1 – Pos: (0.8968, 0.8576) – F: 0.0018

...

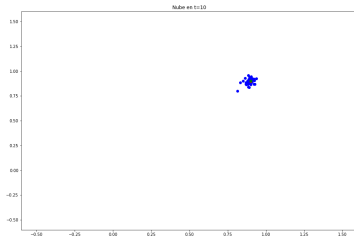
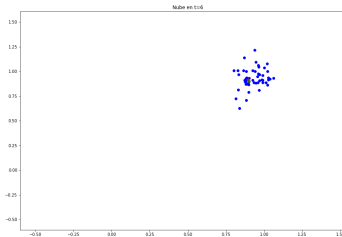
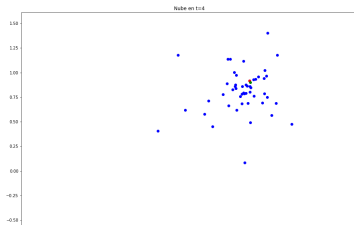
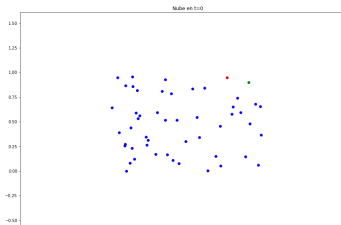
Paso 4 – Pos: (0.8956, 0.9155) – F: 0.0002

...

Paso 10 – Pos: (0.9003, 0.8995) – F: 3.10 e-07

Optimización de enjambre de partículas

Pos. óptima en verde - Mejor pos. hasta t en rojo



Optimización de enjambre de partículas

Aplicaciones

Mathematical Problems in Engineering
Volume 2019, Article ID 8164609, 10 pages
<https://doi.org/10.1155/2019/8164609>

Research Article

Application of Improved Particle Swarm Optimization in Vehicle Crashworthiness

Dawei Gao , Xiangyang Li, and Haifeng Chen

School of Mechanical Engineering, University of Shanghai for Science and Technology, Shanghai 200093, China

Correspondence should be addressed to Dawei Gao;
gddwww1999@163.com

Optimización de enjambre de partículas

Aplicaciones

ORIGINAL RESEARCH ARTICLE

Front. Chem., 12 July 2019 | <https://doi.org/10.3389/fchem.2019.00485>



Modified Particle Swarm Optimization Algorithms for the Generation of Stable Structures of Carbon Clusters, C_n ($n = 3-6, 10$)

 Gourhari Jana¹,  Arka Mitra²,  Sudip Pan³,  Shamik Sural^{4*} and  Pratim K. Chattaraj^{1,5*}

Bibliografía

- Floreano, D. y Mattiussi, C. *Bio-Inspired Artificial Intelligence* (The MIT Press, 2008)
 - Cap. 7: “Collective Systems”.
- Eiben, A.E. y Smith, J.E. *Introduction to Evolutionary Computing* (Springer, 2007).
 - Cap. 3: “Genetic Algorithms”.
- Russell, S. y Norvig, P. *Artificial Intelligence (A Modern Approach) 3rd edition* (Prentice–Hall, 2010).
 - Cap. 4 “Beyond classical search”.
- Mitchell, T.M. *Machine Learning* (McGraw-Hill, 1997)
 - Cap. 9: “Genetic Algorithms”
- Michalewicz, Z. *Genetic Algorithms + Data Structures = Evolution Programs* (Springer, 1999).
 - Cap. 2 “GAs: How Do They Work?”.